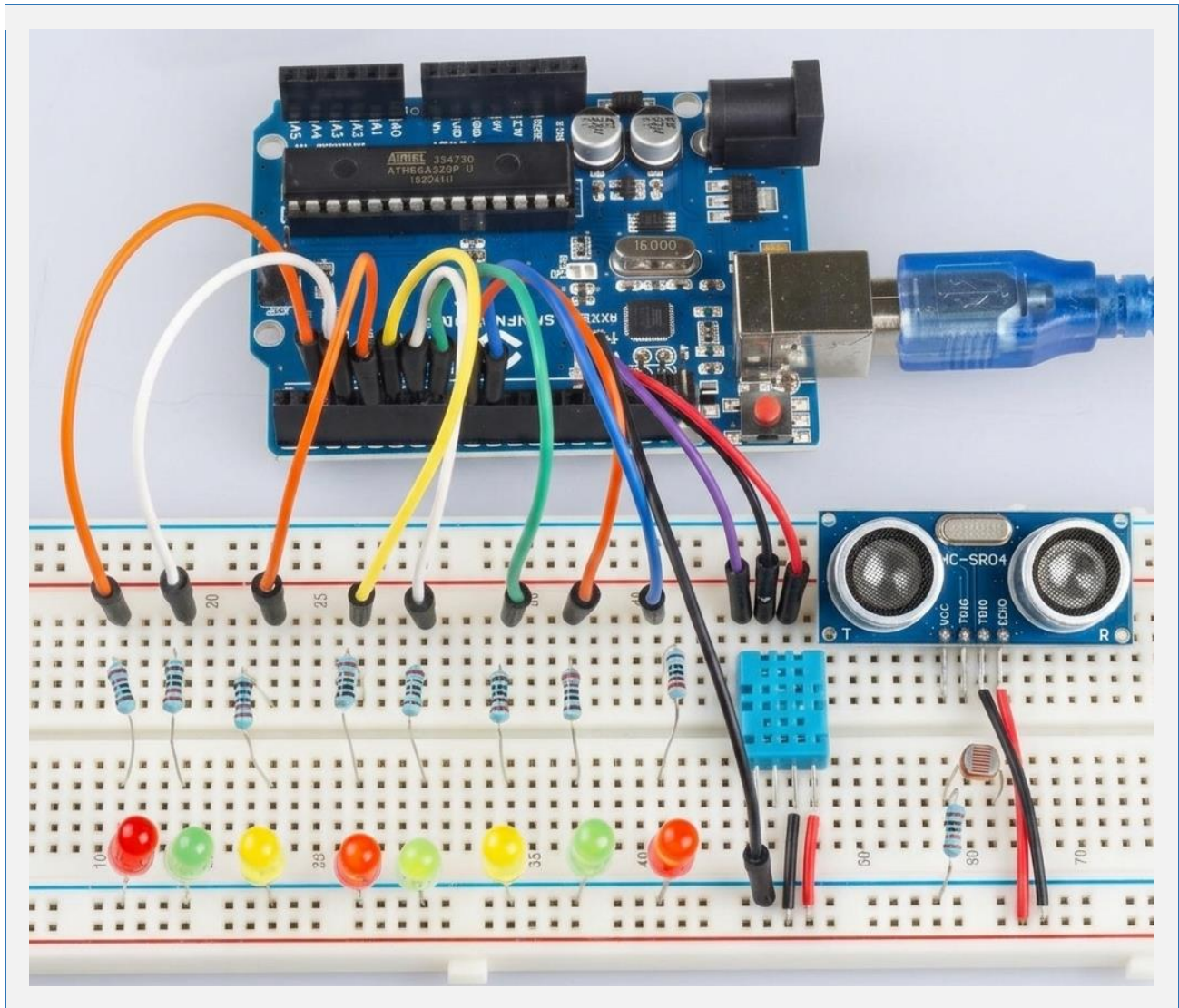


Arduino

Intermediate Curriculum

Grades 5–8

Level 2 STEM Unit | 4-Week Projects & Engineering Challenges



Author: Ahmad Abdullah

© 2026 MATH-O-TRONICS. ALL RIGHTS RESERVED

Premium STEM Curriculum Edition

TABLE OF CONTENTS

Section	Page
Table of Contents	2
Teacher Quick Start Guide	4
Required Prior Knowledge	5
Estimated Duration & Classroom Format	6
Teacher Preparation & Printing Notes	7
Materials Required & Differentiation Support	8
Assessment Overview & Teacher Tips	9
Teacher Introduction	10
Teacher Guide & Implementation Notes	11
Estimated Lesson Timing	12
Common Student Mistakes & Differentiation Strategies	13
Unit Structure at a Glance	14
How to Use This Curriculum	15
Classroom Setup Tips & Assessment Strategy	16
Materials List	17
Optional / Capstone Components	18
NGSS Standards Alignment	19

WEEK 1 — SMART INPUTS & OUTPUTS

Project	Page
Week 1 Overview	20
Project 1 — Push Button Control System	21
Project 2 — LED Reaction Timer	27
Project 3 — Buzzer Challenge: Tones & Patterns	33

WEEK 2 — SENSORS & AUTOMATION

Project	Page
Week 2 Overview	37
Project 4 — Temperature Alert System	38
Project 5 — Light-Controlled Lamp	44
Project 6 — Distance Warning System	49

WEEK 3 — CODING POWER SKILLS

Project	Page
Week 3 Overview	54
Project 7 — Functions Challenge	55
Project 8 — Variables Mission	60
Project 9 — Non-Blocking Timer / millis()	66

WEEK 4 — CAPSTONE ENGINEERING

Project	Page
Project 10 — Mini State Machine Alarm	72
Week 4 — Capstone Engineering Week	79
Capstone Project Planner	81

ASSESSMENT & TEACHER RESOURCES

Section	Page
STEM Lab Participation Rubric	84
Arduino Coding Rubric	85
Engineering Design Rubric	85
Final Capstone Project Rubric	86
Answer Keys	88
General Teaching Notes	104
Teacher Tips and Differentiation	106
Troubleshooting Guide	107
Certificate of Completion	108
About the Author	109

Teacher Quick Start Guide

Welcome to Math-O-Tronics Level 2

Welcome to the Math-O-Tronics Arduino Intermediate Curriculum — Level 2 STEM Unit for Grades 5–8.

This curriculum is designed for students who already understand basic Arduino programming and breadboard building. Through hands-on engineering challenges, students will strengthen their coding, electronics, debugging, and problem-solving skills using real Arduino projects.

Students will explore:



Level 2 Curriculum Scope

- Multi-component circuits combining sensors, LEDs, and buzzers
- Sensors and automation systems (temperature, light, distance)
- Functions and reusable code design
- Non-blocking timing with millis()
- State machines and engineering logic
- Data collection, running averages, and analysis
- Real-world STEM problem solving and the engineering design process

Recommended Grade Levels & Settings



This Curriculum is Designed For



Grades 5–8



STEM Labs & Technology Classes



Makerspaces



Robotics Clubs



Engineering Electives



Computer Science Enrichment Programs

Materials Required

Core Hardware Components (per student pair)

- Arduino Uno (or compatible board)
- USB cable (Type B)
- Full-size breadboard
- LEDs — assorted colors (red, green, yellow)
- Push buttons (momentary)
- Resistors — 220Ω and 10kΩ
- Jumper wires (assorted, 20+ per station)
- Piezo buzzer
- Sensors — TMP36 (temperature), LDR / photoresistor (light), HC-SR04 (ultrasonic distance)

Software

- Arduino IDE — free download at arduino.cc
- Installed on all classroom computers before Day 1
- USB drivers verified and COM port detection confirmed

Differentiation Support

This curriculum includes built-in differentiation strategies for every project:

 Support Track	 Advanced Track
 Pre-wired circuits	 Sensor combinations
 Scaffolded code templates	 Custom function design
 Guided partner roles	 Threshold modifications
 Partial code completion	 Independent engineering extensions

Assessment Overview

Student understanding can be assessed using multiple methods throughout the unit:

 Assessment Methods
 Exit Tickets — quick formative checks at the end of each lesson
 Worksheets — applied thinking collected after each project
 Reflection Responses — higher-order writing assessed with the rubric
 Teacher Observation — note student independence and debugging ability
 Troubleshooting Performance — assess systematic problem-solving skills
 Capstone Engineering Project — performance-based final assessment

Teachers may also assess:

- Collaboration and communication within engineering teams
- Debugging skills and persistence when resolving errors
- Engineering design thinking — planning, building, testing, iterating
- Code readability, logic, and use of comments

Teacher Tip — Building Debugging Habits

 Encourage Independent Troubleshooting First
Before students ask for help, encourage them to check for the most common Arduino errors:
 Loose wires — reseat all component legs and jumper wires firmly
 Reversed LEDs — long leg (anode) must connect toward the resistor/pin
 Incorrect pin numbers — verify code pin assignments match physical wiring
 Missing GND connections — every component needs a complete circuit path
 Incorrect board or port selection — check Tools > Board and Tools > Port
Developing systematic debugging habits is one of the most valuable engineering skills students can build — more important than writing perfect code on the first try.

NGSS Standards Alignment

Next Generation Science Standards — Engineering Design

Standard	Description	Project Alignment
3-5-ETS1-1	Define a simple design problem reflecting a need or want that includes specified criteria for success and constraints on materials, time, or cost.	Capstone Week: Students define criteria and constraints for their smart device. Week 1–3: Students identify what each system must do and under what conditions.
3-5-ETS1-2	Generate and compare multiple possible solutions to a problem based on how well each meets the criteria and constraints of the problem.	Capstone Week: Students compare design options (alarm vs. sensor vs. weather station). Week 3 state machine project requires evaluating multiple logic paths.
3-5-ETS1-3	Plan and carry out fair tests in which variables are controlled and failure points are considered to identify aspects of a model or prototype that can be improved.	All Projects: Students test circuits, record data, analyze failures, and iterate. Troubleshooting sections explicitly practice failure analysis and redesign.

Additional Science & Engineering Practices

Standard	Description	Project Alignment
MS-ETS1-1	Define the criteria and constraints of a design problem with sufficient precision to ensure a successful solution, reflecting society's need for safety.	Week 4 Capstone: Engineering design process from problem definition through testing and presentation.
MS-ETS1-2	Evaluate competing design solutions using a systematic process to determine how well they meet the criteria and constraints of the problem.	Week 3: Students compare blocking vs. non-blocking timing. Week 4: Students justify capstone design choices.
MS-ETS1-3	Analyze data from tests to determine similarities and differences among several design solutions to identify the best solution.	All projects include data recording tables and reflection questions requiring comparative analysis.

Teacher Note on Standards

This curriculum addresses all three NGSS Engineering Design standards at both the 3-5 and MS level. The progression from Week 1 through Week 4 mirrors the engineering design process: understanding the problem → generating solutions → testing and iterating → presenting and communicating results.

PROJECT 1: Push Button Control System

Learning Objectives

- Read digital button input using `digitalRead()`
- Control multiple LEDs using conditional logic
- Apply compound conditions using `&&` and `||`
- Explain how `INPUT_PULLUP` works
- Troubleshoot common button wiring issues
- Interpret Serial Monitor output for debugging

Student Reading Page

In the beginner unit, you learned to blink an LED automatically. Now you are in charge — the LED only responds when YOU decide. A push button gives your Arduino a human interface: a way for a person to communicate with the code.

But there is a challenge. Real buttons are mechanical — when you press one, the metal contacts bounce rapidly before settling, sending dozens of HIGH/LOW pulses in just a few milliseconds. This is called 'button bounce.' Engineers use debouncing to filter out this noise.

In this project, you will build a push button system that controls two LEDs with logic: one LED lights when Button A is pressed, a different LED lights when Button B is pressed, and both light when both are pressed at the same time.

Key Vocabulary

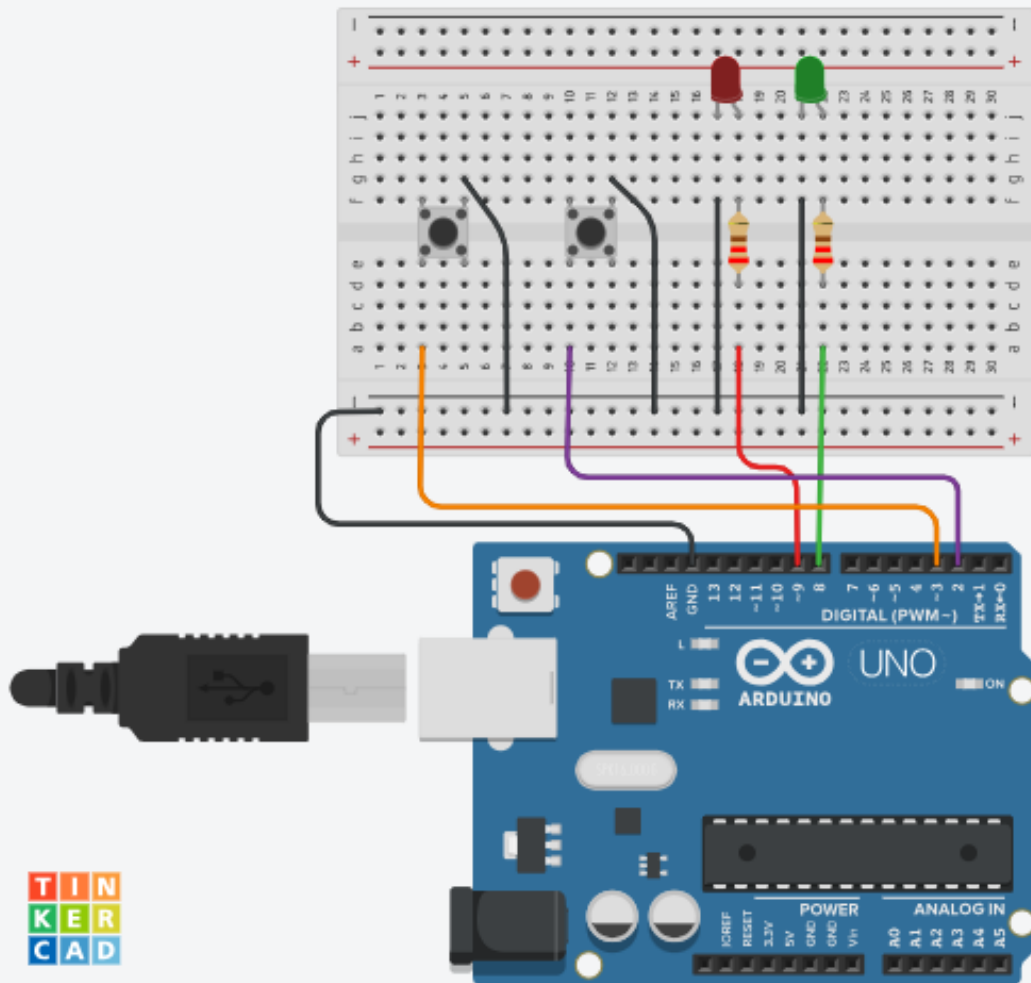
Term	Definition
Debouncing	A technique to ignore the rapid bouncing of button contacts during a press, ensuring only one clean signal is registered.
Compound Conditional	An if statement that uses AND (<code>&&</code>) or OR (<code> </code>) to check two or more conditions at the same time.
<code>digitalRead()</code>	An Arduino function that reads the current state (HIGH or LOW) of a digital pin.
<code>INPUT_PULLUP</code>	A pin mode that enables an internal resistor holding the pin HIGH. The pin reads LOW when the button connects it to GND.
State	The current condition of a system at any point in time (e.g., 'Button A pressed, Button B not pressed').
Boolean Logic	Decision-making using true/false values combined with AND, OR, and NOT operations.

Materials Needed

• 1 × Arduino Uno • 2 × Push buttons • 2 × LEDs (different colors) • 2 × 220Ω resistors • 1 × Breadboard • Jumper wires • 1 × USB cable

Circuit Build Instructions

1. Connect Button A: one leg to Digital Pin 2, other leg to GND.
2. Connect Button B: one leg to Digital Pin 3, other leg to GND.
3. Connect LED 1 (Green): long leg through 220Ω resistor to Digital Pin 8, short leg to GND.
4. Connect LED 2 (Red): long leg through 220Ω resistor to Digital Pin 9, short leg to GND.
5. No external pull-up resistors needed — code uses INPUT_PULLUP for both buttons.



Arduino Code — Push Button Control

```
// Project 1: Push Button Control System
// Level 2 | Math-O-Tronics | Author: Ahmad Abdullah

const int buttonA = 2;
const int buttonB = 3;
const int ledGreen = 8;
const int ledRed = 9;

void setup() {
  pinMode(buttonA, INPUT_PULLUP);
```

```

pinMode(buttonB, INPUT_PULLUP);
pinMode(ledGreen, OUTPUT);
pinMode(ledRed, OUTPUT);
Serial.begin(9600);
}

void loop() {
  bool btnA = (digitalRead(buttonA) == LOW); // true when pressed
  bool btnB = (digitalRead(buttonB) == LOW);

  // Both buttons pressed at same time
  if (btnA && btnB) {
    digitalWrite(ledGreen, HIGH);
    digitalWrite(ledRed, HIGH);
    Serial.println("BOTH pressed!");
  }
  // Only Button A
  else if (btnA) {
    digitalWrite(ledGreen, HIGH);
    digitalWrite(ledRed, LOW);
    Serial.println("Button A");
  }
  // Only Button B
  else if (btnB) {
    digitalWrite(ledGreen, LOW);
    digitalWrite(ledRed, HIGH);
    Serial.println("Button B");
  }
  // Nothing pressed
  else {
    digitalWrite(ledGreen, LOW);
    digitalWrite(ledRed, LOW);
  }

  delay(50); // Simple debounce delay
}

```

Student Worksheet — Project 1

1. What does the && operator mean? Give an example from the code above.

2. Why does the code use INPUT_PULLUP instead of a 10kΩ resistor? What are the benefits?

3. If you pressed Button A while Button B was already held down, which condition runs first? Why does the order of if/else matter?

Data Recording Table

Button Combination	Green LED	Red LED	Serial Monitor Output	Correct? Y/N

Reflection — Higher-Order Thinking

4. Design a third button that turns OFF both LEDs regardless of any other button state. Where in the if/else chain would you place it, and why?

Extension Challenge

Add a third LED (Yellow) that blinks at 2Hz whenever NEITHER button is pressed. Use `millis()` instead of `delay()` so button presses are never missed.

Exit Ticket

What does the expression `(btnA && btnB)` evaluate to when both buttons are pressed? Circle: TRUE / FALSE

Name one real-world device that uses compound button logic: _____



ARDUINO CODING RUBRIC

Use for Arduino programming assignments, sketch development, and code reviews.

Criteria	4 — Excellent	3 — Proficient	2 — Developing	1 — Beginning
Code Accuracy	Code worked correctly with no major errors	Code worked with minor errors	Code partially worked but required major fixes	Code did not function correctly
Logic & Problem Solving	Demonstrated strong understanding of program logic and conditions	Demonstrated good understanding of most concepts	Showed partial understanding of logic	Demonstrated limited understanding
Use of Functions & Variables	Used functions and variables effectively and correctly	Used most functions and variables correctly	Used some programming structures incorrectly	Rarely used structures correctly
Code Organization	Code was clean, organized, and easy to read	Code was mostly organized and readable	Code organization was inconsistent	Code was difficult to follow
Comments & Documentation	Added meaningful comments explaining program behavior	Included some helpful comments	Included minimal comments	No comments included



ENGINEERING DESIGN RUBRIC

Use for design challenges, prototype builds, and iterative improvement projects.

Criteria	4 — Excellent	3 — Proficient	2 — Developing	1 — Beginning
Planning & Design	Created detailed plans with thoughtful engineering decisions	Created clear project plans	Planning was incomplete or unclear	Minimal planning evident
Creativity & Innovation	Demonstrated highly creative ideas and modifications	Included some creative elements	Limited creativity shown	Project copied example with no modifications

ABOUT THE AUTHOR

Founder of Math-O-Tronics

Ahmad Abdullah

Ahmad Abdullah is a STEM educator with a passion for combining mathematics, technology, and hands-on engineering to create meaningful learning experiences for students.



Bachelor of Science in Mathematics / Education — Lebanese International University



Certified Robotics Trainer — Global Board Lebanon

Professional Background & STEM Skills

Ahmad has a strong academic background in Mathematics and Technology. In addition to his formal education, he continuously develops his skills in:

- electronics
- programming
- robotics
- Arduino systems
- educational technology

He believes in lifelong learning and regularly explores new teaching tools, STEM strategies, and project-based learning methods to help students develop confidence, curiosity, and critical thinking skills.

Ahmad is especially passionate about helping students understand mathematics through clear, logical instruction connected to real-world applications.

STEM Activities & Interests

- building electronics projects
- coding with Arduino
- designing robotics activities
- creating classroom-ready STEM resources

His mission through Math-O-Tronics is to provide practical, classroom-tested materials that make teaching easier and learning more engaging for students.